

אפליקציית ווב להתאמת שגרת טיפוח אישית – PersonalSkin

שם: לירון כהן

ת"ז: 331536920

PersonalSkin – נושא העבודה: אפליקציית ווב להתאמת שגרת טיפוח פנים אישית

שם בית הספר ועיר: תיכון בליך ר"ג

מורה מלווה: צבי שרן

מועד הגשה: מאי 2026

תוכן עניינים {#} תוכן-עניינים {#}

פרק א' – ייזום {#} פרק-א' – ייזום)

תיאור ראשוני של המערכת {#} תיאור-ראשוני-של-המערכת)

הגדרת הלקוח {#} הגדרת-הלקוח)

הגדרת יעדים/מטרות {#} הגדרת-יעדים/מטרות)

בעיות, תועלות וחשבוניות {#} בעיות-תועלות-וחשבוניות)

האם צפויים קשיים או מגבלות בהגדרת המערכת {#} האם-צפויים-קשיים-או-מגבלות-בהגדרת-המערכת)

תיחום הפרויקט {#} תיחום-הפרויקט)

פרק ב' – אפיון {#} פרק-ב' – אפיון)

פירוט המערכת {#} פירוט-המערכת)

יכולות המערכת {#} יכולות-המערכת)

פירוט הבדיקות {#} פירוט-הבדיקות)

תכנון לוח זמנים {#} תכנון-לוח-זמנים)

ניהול סיכונים {#} ניהול-סיכונים)

פרק ג' – ניתוח {#} פרק-ג' – ניתוח)

יכולות המערכת – פירוט פונקציונלי {#} יכולות-המערכת – פירוט-פונקציונלי)

פרק ד' – עיצוב {#} פרק-ד' – עיצוב)

תיאור הארכיטקטורה של המערכת המוצעת {#} תיאור-הארכיטקטורה-של-המערכת-המוצעת)

תיאור הטכנולוגיה הרלוונטית (#תיאור-הטכנולוגיה-הרלוונטית)
תיאור מודולים בהם נעשה שימוש (#תיאור-מודולים-בהם-נעשה-שימוש)
מודולים מיובאים (#מודולים-מיובאים)
מודולים שלי שבשימוש (#מודולים-שלי-שבשימוש)
תיאור סביבת הפיתוח (#תיאור-סביבת-הפיתוח)
תיאור האלגוריתמים המרכזיים בפרויקט (#תיאור-האלגוריתמים-המרכזיים-בפרויקט)
תיאור מסכי הפרויקט (#תיאור-מסכי-הפרויקט)
פרוטוקול התקשורת (#פרוטוקול-התקשורת)
מבני נתונים (#מבני-נתונים)
פרק ה' – הקוד (#פרק-ה-הקוד)
פרק ו' – בדיקות (#פרק-ו-בדיקות)
פרק ז' – מדריך למשתמש (#פרק-ז-מדריך-למשתמש)
פרק ח' – מבט אישי (#פרק-ח-מבט-אישי)
פרק ט' – ביבליוגרפיה (#פרק-ט-ביבליוגרפיה)
נספח – הקוד (#נספח-הקוד)

פרק א' – ייזום

תיאור ראשוני של המערכת

תקציר כולל ורצינות הפרויקט

בעולם הטיפוח, אנשים רבים מתקשים להרכיב שגרת טיפוח פנים מתאימה לסוג העור שלהם, לגילם, לרגישויות ולתקציבם. מרבית הפתרונות הקיימים הם מאמרים כלליים או ייעוץ יקר אצל קוסמטיקאית. היא אפליקציית ווב שמאפשרת למשתמשים למלא שאלון פרופיל עור מפורט ולקבל שגרת PersonalSkin או לפי API, כאשר מוגדר מפתח (OpenAI) טיפוח בוקר וערב מותאמת אישית – שנוצרת באמצעות מודל שפה מנגנון גיבוי מבוסס כללים כאשר אין מפתח או כאשר הקריאה נכשלת. בנוסף, קיים דף קהילה בו משתמשים יכולים לשתף את הפרופיל והשגרה שלהם ולראות שיתופים של אחרים.

ההמלצות מיועדות להסבר כללי בלבד ואינן תחליף לייעוץ. RTL הממשק והתוכן מוצגים בעברית בכיוון רפואי או מקצועי.

מה המוצר המוגמר אמור לבצע

המוצר המוגמר אמור לאפשר לכל משתמש

- להירשם ולהתחבר למערכת בצורה מאובטחת (סיסמאות מוצפנות, סשן אישי)
- למלא שאלון מקיף של פרופיל עור (סוג עור, גיל, חששות, רגישויות, מטרות, תקציב, זמן זמין, אקלים, (איפור, אקספוליאציה, ניסיון רטינול, הריון/הנקה ועוד, SPF, הרגלי
- מעוצב Markdown לקבל שגרת טיפוח בוקר וערב מותאמת אישית שנוצרת בזמן אמת ומוצגת כמסמך
 - לעדכן את השאלון בכל עת ולקבל שגרה מחודשת (עם שמירת היסטוריית גרסאות)
- לשתף את הפרופיל והשגרה בדף קהילה עם שם תצוגה, ולצפות בשיתופים של משתמשים אחרים
 - לתת לייק/דיסלייק על שיתופים בקהילה

למה בחרתי את הפרויקט ואתגרים צפויים

בחרתי את הפרויקט הזה מכיוון ששילוב בין פיתוח ווב מלא (צד שרת וצד לקוח) לבין שימוש בבינה מלאכותית הוא נושא שמאוד מעניין אותי ורלוונטי לעולם של היום. רציתי לבנות מערכת שרת-לקוח שלמה עם מסד שיוצרת AI נתונים, אימות משתמשים, אבטחת מידע, וחווית משתמש מלאה – ובנוסף לחבר אותה ליכולת תוכן מותאם אישית. האתגרים שצפיתי היו: עיצוב שאלון מפורט מספיק כדי שהמודל ייתן תוצאות טובות, ובניית ממשק AI, שיתן שגרה סבירה גם ללא Fallback הטמעת מנגנון OpenAI-ייעיל ל Prompt כתיבת שנראה מקצועי ונוח RTL משתמש בעברית

הגדרת הלקוח

למי מיועדת המערכת ומי הולך להשתמש בה

המערכת מיועדת לשני סוגי לקוחות עיקריים:

1. אנשים שרוצים שגרת טיפוח אישית – משתמשים שמתעניינים בטיפוח פנים אבל לא יודעים איפה להתחיל, או שרוצים המלצה מותאמת לסוג העור והצרכים הייחודיים שלהם ללא צורך ביעוץ יקר
2. קהילת טיפוח – משתמשים שרוצים לראות מה אחרים עם פרופיל עור דומה משתמשים, לשתף את השגרה שלהם ולקבל השראה מאחרים

הגדרת יעדים/מטרות

מה המטרות המרכזיות של המערכת המוצעת

3. לספק שגרת טיפוח מותאמת אישית על סמך נתוני העור והצרכים של המשתמש
4. ליצירת המלצות מפורטות ומותאמות, עם מנגנון גיבוי כללי שמבטיח (OpenAI) לנצל בינה מלאכותית. שהמערכת תמיד תיתן תוצאה
5. ליצור חוויית משתמש נקייה ומובנת בעברית
6. לאפשר שיתוף ולמידה הדדית באמצעות דף קהילה
7. (CSRF, XSS), אימות, הצפנת סיסמאות) להציג מערכת שרת-לקוח מלאה ומאובטחת

בעיות, תועלות וחסיכות

מה הבעיות אותן אנו מנסים לפתור

הבעיה הראשונה היא שמידע על טיפוח נגיש בעיקר בצורה כללית – מאמרים שמתייחסים לקטגוריה אחת ("עור שומני") (בלי להתחשב בעשרות פרמטרים נוספים כמו אקלים, רגישויות, תקציב וזמן. הבעיה השנייה היא שיעוץ אישי (קוסמטיקאית/רופא עור) (הוא יקר ולא תמיד נגיש. הבעיה השלישית היא שמשתמשים לא יודעים אם השגרה שלהם טובה – אין להם דרך פשוטה להשוות לאחרים עם פרופיל דומה

מה התועלות שסביר לצפות מהמערכת

8. קבלת שגרת טיפוח מותאמת תוך שניות, ללא עלות

9. התאמה לפרמטרים אישיים מפורטים 18 שדות בשאלון).
10. יכולת לעדכן ולהתאים את השגרה לאורך זמן (שמירת גרסאות).
11. למידה מהקהילה – צפייה בשגרות של אחרים עם פרופיל דומה.
12. חיסכון בזמן וכסף (אין צורך ביועץ).

אילו שירותים המערכת תיתן

(AI) הרשמה והתחברות מאובטחת, מילוי שאלון פרופיל עור מפורט, יצירת שגרת בוקר/ערב מותאמת אישית או כללים, (עדכון שאלון ויצירת שגרה מחודשת, צפייה בהיסטוריית גרסאות, שיתוף שגרה ופרופיל בדף קהילה, צפייה בשיתופים של אחרים, וכן לייק/דיסלייק על שיתופים

השוואת העבודה עם פתרונות ויישומים קיימים

מאמרים ומדריכים כלליים שמחולקים לפי סוג עור אך לא מותאמים – **SkinCare Routine by Dermstore** (<https://www.dermstore.com>) לפרמטרים אישיים מפורטים. קישור

שירותים בתשלום שמייצרים מוצרים מותאמים. הם דורשים מנוי חודשי – **Curology / Proven Skincare** (<https://curology.com>) ו-**Proven** (<https://www.provenskincare.com>) קישור ל

מתאים ולא שומר Prompt ישיר – ניתן לשאול מודל שפה ישירות, אך הדבר דורש ניסוח **ChatGPT** היסטוריה, קהילה, או ממשק ייעודי

נבדלת מכל הפתרונות הללו בכך שהיא אפליקציה חנימית עצמאית, בעברית, עם שאלון מובנה, **PersonalSkin**, היסטוריית גרסאות, ודף קהילה – והכול בממשק אחד נוח

האם צפויים קשיים או מגבלות בהגדרת המערכת

האם מדובר בטכנולוגיה חדשה ובלתי מוכרת

Flask (Python), SQLAlchemy, לא. כל הטכנולוגיות בהן נעשה שימוש הן מוכרות ומבוססות החידוש הוא בשילוב ביניהן ליצירת מוצר שלם עם חוויית משתמש HTML/CSS, ו-OpenAI API. קוהרנטית

האם קיימים סייגים בהגדרת המערכת

13. ההמלצות אינן תחליף לייעוץ רפואי – המערכת מציגת זאת בפני המשתמש.
14. כאשר אין מפתח או שיש כשל, מופעל מנגנון – OpenAI של API-איכות השגרה תלויה בזמינות ה Fallback פשוט יותר.
15. (השרת עלול "להירדם" לאחר חוסר פעילות Render בתכנית חנימית של.
16. מתאים לפיתוח אך עבור סביבת ייצור עם הרבה משתמשים מקביליים מומלץ SQLite-מסד הנתונים ב PostgreSQL.

תיחום הפרויקט

התחומים בהם הפרויקט עוסק

הפרויקט עוסק בתחומים הבאים:

17. מדפדפן (לקוח). (תקשורת מבוססת HTTP שמטפל בבקשות Flask (Python) מערכת שרת-לקוח – שרת HTTP/HTTPS.

18. בייצור. שמירת משתמשים, פרופילי עור, שגרות, גרסאות, PostgreSQL, בפתוח SQLite – מסדי נתונים. שיתופים ותגובות.
19. XSS (Bleach), מניעת (Flask-WTF) CSRF הגנת (Werkzeug hash) אבטחת מידע – הצפנת סיסמאות. (Audit Log) מדיניות סיסמה חזקה, תיעוד אירועי אבטחה
20. ליצירת תוכן מותאם לפי נתוני השאלון OpenAI Chat Completions API-בינה מלאכותית – שימוש ב.
21. (Queue + Worker Thread) ייעודי לכתובת לוגים אסינכרונית Thread – מערכת הפעלה ותהליכים. Windows ופיתוח על Linux (Render) פריסה על
22. עיצוב רספונסיבי, מסכים מרובים) בית, הרשמה, , RTL, בעברית HTML/CSS – ממשק משתמש. (התחברות, שאלון, שגרה, קהילה

מה הפרויקט אינו כולל:

- אין רכיב חומרה.
- אין יצירת מוצרים פיזיים.
- אין ייעוץ רפואי.
- אין תמיכה בשפות נוספות (עברית בלבד).

פרק ב' – אפיון

פירוט המערכת

בעולם של ימינו, ענף הטיפוח והקוסמטיקה גדל מדי שנה, ובמקביל גובר בלבול הצרכנים לגבי מה באמת מתאים לִפְנֵם. המידע באינטרנט מוצף ולא תמיד אמין, ויש פער גדול בין מי שיכול לשלם ליועץ טיפוח אישי מגשרת על הפער: המשתמש ממלא שאלון מפורט של 18 שדות הנוגעים לסוג PersonalSkin. לבין מי שלא העור, הרגלים, אורח חיים ותקציב, והמערכת מייצרת עבורו שגרה אישית בפורמט ברור ומובנה. כאשר מוגדר כאשר אין מפתח או שיש (GPT-4o-mini) השגרה נוצרת על ידי מודל שפה מתקדם, OpenAI של API מפתח כשל, מופעל מנגנון גיבוי מבוסס כללים שמבטיח שהמשתמש תמיד יקבל תוצאה. בנוסף, דף הקהילה מאפשר שיתוף ולמידה הדדית.

יכולות המערכת

23. הרשמה למערכת עם אימייל וסיסמה (כולל מדיניות סיסמה חזקה).
24. (remember-me סשן עם) התחברות מאובטחת.
25. איפוס סיסמה (שכחתי סיסמה).
26. מילוי שאלון פרופיל עור מפורט (18 שדות).
27. יצירת שגרת בוקר מותאמת אישית.
28. יצירת שגרת ערב מותאמת אישית.
29. ליצירת שגרה חכמה (OpenAI) AI-שימוש ב.
30. לא זמין AI-יצירת שגרה לפי כללים כאשר ה – Fallback מנגנון.
31. שמירת היסטוריית גרסאות של שגרות.
32. עדכון השאלון ויצירת שגרה מחודשת בכל עת.
33. שיתוף פרופיל ושגרה בדף קהילה (עם שם תצוגה).
34. צפייה בפייד שיתופים של משתמשים אחרים (גם ללא התחברות).
35. לייק /דיסלייק על שיתופים בקהילה.

36. Audit Log) תיעוד אירועי אבטחה אסינכרוני

37. נקי ונוח RTL ממשק בעברית

פירוט הבדיקות

איך מתכננים לבדוק	מה אמורה לבדוק	שם הבדיקה	מספר
ניגשים לדף הרשמה, ממלאים את כל השדות, לוחצים הרשמה — מצפים להפניה לשאלון	משתמש חדש יכול להירשם עם אימייל וסיסמה תקינים	הרשמה תקינה	1
מנסים להירשם עם סיסמאות שונות שחסרות דרישה — מצפים להודעת שגיאה	מערכת דוחה סיסמה שלא עומדת במדיניות 8 תווים, (אות, מספר, תו מיוחד)	סיסמה חלשה	2
מנסים להתחבר עם סיסמה שגויה — מצפים להודעת "אימייל או סיסמה שגויים"	מערכת מזהה אימייל/סיסמה לא נכונים ולא מכניסה	התחברות שגויה	3
מתחברים עם פרטים נכונים — מצפים להגיע לדף שאלון	משתמש רשום מצליח להתחבר ומופנה לשאלון	התחברות נכונה	4
ממלאים את כל 18 השדות ושולחים — מצפים לראות שגרת בוקר וערב	שאלון תקין נשמר ושגרה נוצרת	מילוי שאלון מלא	5
ממלאים רק חלק מהשדות ושולחים — מצפים להודעת שגיאה	מערכת לא מקבלת שאלון שלא מולאו בו שדות חובה	שאלון חלקי	6
מוודאים ש- <code>'OPENAI_API_KEY'</code> מוגדר, שולחים שאלון, בודקים ש- <code>'used_openai=True'</code>	OpenAI כאשר יש מפתח AI השגרה נוצרת דרך	AI יצירת שגרה עם	7
מסירים את המפתח, שולחים שאלון, בודקים ש- <code>'used_openai=False'</code> ושגיאה	כאשר אין מפתח — מנגנון הגיבוי יוצר שגרה	AI ללא Fallback	8
לוחצים על "שיתוף", ממלאים שם תצוגה — מצפים לראות פוסט בפייד	משתמש עם שגרה יכול לפרסם שיתוף	שיתוף בקהילה	9
לוחצים לייק, מוודאים שהוא מופיע; לוחצים שוב — מצפים שיתבטל	משתמש יכול לתת ולבטל תגובה	לייק/דיסלייק	10
ישיר ללא POST שולחים מצפים לדחייה — token (400/403)	CSRF token טופס ללא נדחה	CSRF הגנת	11
מבצעים פעולות שונות ובודקים את תוכן <code>'logs/audit.log'</code>	אירועי התחברות/שגיאה נרשמים בקובץ	Audit Log	12

תכנון לוח זמנים

הערות	זמן סיום בפועל	זמן התחלה בפועל	זמן סיום מתוכנן	זמן התחלה מתוכנן	פעילות
					מסמך ייזום ואפיון
					Flask מבנה בסיסי + DB
					מודול הרשמה + התחברות
					שאלון פרופיל עור
					יצירת AI – שירות שגרה
					Fallback מנגנון
					דף הצגת שגרה

דף קהילה + שיתוף					
לייק/דיסלייק + תגובות					
Audit Logger אסינכרוני					
CSS + RTL עיצוב					
גרסאות שגרה (היסטוריה)					
Render-פריסה ל					
בדיקות סופיות					
הגשת תיק פרויקט					

ניהול סיכונים

תאריך	מה בוצע בפועל	דרכי התמודדות	רמת הסיכון	פירוט הסיכון	הסיכון
	<code>`_build_fallback_routine`</code> נבנה תמיד מחזיר <code>— routine_ai.py`-ב</code> שגרה	Fallback בניית מנגנון 1) (מבוסס כללים 2 try/except סביב כל קריאה	קשה	אם המודל לא זמין, אין מפתח, או תשובה פגומה המשתמש — לא יקבל שגרה	API-כשל ב** OpenAI של
	עם Hash <code>`generate_password_hash`;</code> ולידציה: 8+ תווים, אות, מספר, תו מיוחד	Hash-שימוש ב 1) (Werkzeug) 2) מדיניות חוזק סיסמה	קשה	אם סיסמאות נשמרות כטקסט — פגיעה חמורה בפרטיות	אבטחת ** סיסמאות
	attribute/מסנן כל תגית Bleach שלא ברשימה המותרת	whitelist עם Bleach 1) של תגיות 2 (אי-הצגת HTML גולמי	בינוני	תוכן Markdown זדוני עלול להריץ JavaScript	התקפת ** XSS
	<code>WTF_CSRF_ENABLED=True;</code> <code>`hidden_tag0`</code> כל טופס כולל	עם Flask-WTF 1) CSRF token 2) <code>`validate_on_submit0`</code>	בינוני	משתמש זדוני שולח טופס בשם משתמש אחר	התקפת ** CSRF
		תכנון בשלבים 2) 1) עבודה מראש על חלקים מוכרים	בינוני	עיכוב בגלל לימוד טכנולוגיה חדשה	אי עמידה ** בלוחות הזמנים
	Postgres מגדיר <code>`render.yaml`</code> URI מנוהל; הקוד מזהה ומתאים	PostgreSQL-מעבר ל 1) (בייצור 2 DATABASE_URL env var	קל	לא SQLite תומך בכתיבה מקבילית	בעיית בסיס ** נתונים בייצור
	ניתן לשדרג; README-תועד ב בעתיד	שדרוג לתוכנית 1) -בתשלום 2 (שימוש ב Health Check	קל	בתוכנית חנימית השרת ישן "אחרי 15" דק' חוסר פעילות	שרת ** Render נרדם

פרק ג' – ניתוח

יכולות המערכת – פירוט פונקציונלי

הרשמה למערכת. 38.

מהות היכולת

היכולת מאפשרת למשתמש חדש ליצור חשבון במערכת.

תהליכים ויכולות

- המשתמש ניגש לדף הרשמה וממלא: אימייל, שם פרטי (אופציונלי), (שם משפחה) אופציונלי, (סיסמה ואימות סיסמה).
- הטופס עובר ולידציה: אימייל תקני, סיסמה עומדת במדיניות (8+ תווים, אות, מספר, תו מיוחד), (אימייל לא רשום כבר).
- ונשמרת במסד הנתונים (Hash) הסיסמה מוצפנת.
 - DB-ונשמר ב User נוצר אובייקט.
 - המשתמש מחובר אוטומטית ומופנה לשאלון.
 - Audit Log-נרשם ב auth.register_success אירוע.

אובייקטים בהם היכולת עושה שימוש

- User (מודל)
- RegisterForm (טופס WTForms)
 - db (SQLAlchemy)
 - login_user (Flask-Login)
 - Audit Logger

התחברות למערכת. 39.

מהות היכולת

היכולת מאפשרת למשתמש רשום להיכנס לחשבון.

תהליכים ויכולות

- המשתמש מכניס אימייל וסיסמה.
- check_password_hash ומשווה סיסמה עם DB-המערכת מאתרת את המשתמש ב.
 - login_user(remember=True) מתחברת עם last_login אם נכון — מעדכנת.
 - Audit-ב auth.login_failed אם שגוי — מציגה הודעת שגיאה ורושמת.
 - אם קיים, אחרת לשאלון URL-next-הפנייה ל.

אובייקטים בהם היכולת עושה שימוש

- User
- LoginForm

- db
- login_user
- Audit Logger

איפוס סיסמה. 40.

מהות היכולת

היכולת מאפשרת למשתמש ששכח סיסמה לאפס אותה.

תהליכים ויכולות

- המשתמש מכניס אימייל וסיסמה חדשה (עם אימות).
- המערכת מוודאת שהאימייל קיים ושהסיסמה החדשה עומדת במדיניות.
 - הסיסמה החדשה מוצפנת ונשמרת.
 - נרשם auth.password_reset_success אירוע.
 - הפנייה לדף התחברות.

אובייקטים בהם היכולת עושה שימוש

- User
- ResetPasswordForm
 - db
 - Audit Logger

מילוי שאלון פרופיל עור. 41.

מהות היכולת

היכולת מאפשרת למשתמש להזין את כל נתוני העור והצרכים שלו.

תהליכים ויכולות

- רגישויות, מטרות, תקציב, דקות, (multi-select) הצגת טופס עם 18 שדות: סוג עור, טווח גיל, חששות אקספוליאציה, ניסיון רטינול, SPF, בבוקר/ערב, אקלים, מי ברז, חשיפה לשמש, תדירות איפור הריון/הנקה, שימוש בחומרים פעילים, הערות.
 - אם יש פרופיל קיים — השדות ממולאים מראש.
 - לאחר שליחה: ולידציה של כל השדות.
 - SkinProfile ב JSON-שמירת התשובות כ.
 - לתצוגה (summary_label) יצירת תווית סיכום.
 - generate_routine() קריאה ל.

אובייקטים בהם היכולת עושה שימוש

- SkinProfile (מודל)
- SkinQuestionnaireForm (WTForms)

- Routine
- RoutineVersion
- routine_ai service
 - db
- Audit Logger

42. (AI) יצירת שגרת טיפוח

מהות היכולת

היכולת מייצרת שגרת בוקר וערב מותאמת אישית באמצעות מודל שפה

תהליכים ויכולות

- OPENAI_API_KEY בדיקה אם קיים
- (user message) נתוני השאלון + (system) הוראות מערכת: Prompt בניית
- OpenAI Chat Completions API (מודל gpt-4o-mini, temperature=0.5) - קריאה ל
 - פיצול התשובה לשני חלקים: בוקר וערב (לפי כותרות ##ומילות מפתח)
 - Fallback — אם הפיצול נכשל
 - (morning_text, evening_text, used_openai=True) החזרת

אובייקטים בהם היכולת עושה שימוש

- OpenAI Client
 - generate_routine()
- _format_answers_for_prompt()
 - _split_morning_evening()

43. Fallback יצירת שגרה – מנגנון

מהות היכולת

לא זמין — המערכת מייצרת שגרה מובנית לפי כללים AI-כאשר ה

תהליכים ויכולות

- קריאת תשובות: סוג עור, חששות, רגישויות, תקציב
- מובנה עם שלבים ממוספרים לבוקר ולערב Markdown בניית טקסט
 - (acne/dryness/wrinkles) התאמת ההמלצות לפי חששות
 - (הערות בטיחות) הריון, רטינוול
- (morning_text, evening_text, used_openai=False) החזרת

אובייקטים בהם היכולת עושה שימוש

- _build_fallback_routine()
 - answers dict

צפייה בשגרה 44.

מהות היכולת

מעוצב Markdown הצגת השגרה למשתמש בפורמט

תהליכים ויכולות

- DB-מה SkinProfile ו-Routine שליפת
- markdown_safe (Markdown → HTML → Bleach) עם פילטר HTML הצגה בתבנית
- הצגת מספר גרסה נוכחי

אובייקטים בהם היכולת עושה שימוש

- Routine
- SkinProfile
- RoutineVersion
- render_markdown_safe (utils)

שמירת היסטוריית גרסאות 45.

מהות היכולת

כל יצירת שגרה חדשה נשמרת כגרסה ממוספרת

תהליכים ויכולות

- RoutineVersion) בדיקת מספר הגרסה האחרונה
- RoutineVersion יצירת רשומת version_number+1 עם
- JSON, ותשובות, used_openai, שמירת טקסט בוקר/ערב

אובייקטים בהם היכולת עושה שימוש

- RoutineVersion (מודל)
- db

שיתוף בקהילה 46.

מהות היכולת

היכולת מאפשרת למשתמש לפרסם את הפרופיל והשגרה שלו בדף הקהילה

תהליכים ויכולות

- המשתמש ממלא שם תצוגה
- ושל שגרת בוקר/ערב (כל שדות השאלון + summary) המערכת מרכיבה עותק טקסט של הפרופיל
- DB-ונושם ב CommunityShare נוצר אובייקט
- נרשם community.share_success אירוע

אובייקטים בהם היכולת עושה שימוש

- CommunityShare (מודל)
 - ShareForm
 - SkinProfile
 - Routine
 - db
- Audit Logger

צפייה בפיד הקהילה. 47.

מהות היכולת

הצגת שיתופים של משתמשים אחרים, כולל ספירת לייקים

תהליכים ויכולות

- שליפת כל השיתופים (ממוינים לפי תאריך, חדשים ראשונים)
- (SQL aggregation) חישוב צבירת לייקים/דיסליקים לכל שיתוף
- (like? כבר עשה) אם המשתמש מחובר — הצגת מצב התגובה שלו
 - feed.html הצגה בתבנית

אובייקטים בהם היכולת עושה שימוש

- CommunityShare
- CommunityReaction
- db.session.query (aggregation)

לייק / דיסלייק. 48.

מהות היכולת

משתמש מחובר יכול להגיב בלייק או דיסלייק על שיתוף

תהליכים ויכולות

- אם לא קיימת תגובה — נוצרת חדשה
- (Toggle) אם כבר קיימת אותה תגובה — נמחקת
 - אם קיימת תגובה הפוכה — מתעדכנת
- נרשם community.reaction_changed אירוע

אובייקטים בהם היכולת עושה שימוש

- CommunityReaction (מודל)
 - ReactionForm
 - db

- Audit Logger

49. (Audit Log) תיעוד אירועי אבטחה

מהות היכולת

תיעוד כל האירועים הרגישים בקובץ לוג, באופן אסינכרוני

תהליכים ויכולות

- רץ ברקע (audit-log-worker) ייעודי Thread.
- (main thread לא חוסם את ה) Queue-כל אירוע נשלח ל
- (RotatingFileHandler, 2MB, 5 backups, עד 2 logs/audit.log לקובץ JSON שולף מהתור וכותב Worker-ה

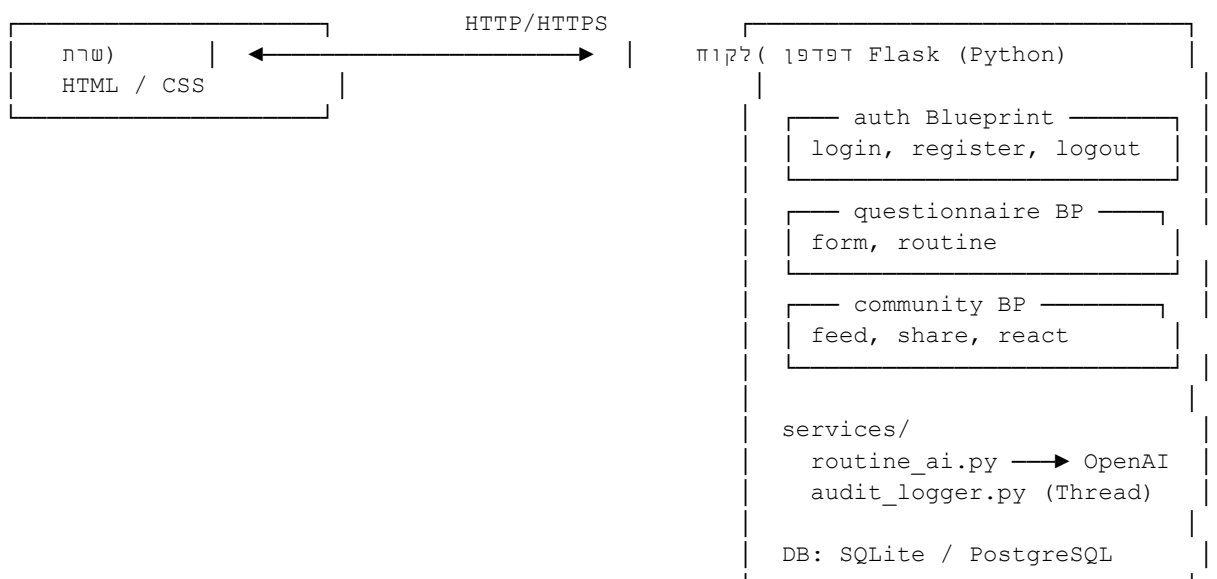
אובייקטים בהם היכולת עושה שימוש

- Queue (threading)
- Thread
- RotatingFileHandler (logging)
- log_audit_event()

פרק ד' – עיצוב

תיאור הארכיטקטורה של המערכת המוצעת

קלאסית (Client-Server) המערכת בנויה בארכיטקטורת שרת-לקוח



רכיבים

- שמגיעות מהשרת. כל אינטראקציה היא HTML/CSS דפדפן אינטרנט — מציג תבניות (Client): לקוח HTTP (GET/POST) בקשת
- לוגיקה עסקית, אימות משתמשים, (Routing), מטפלת בניית Flask אפליקציית שרת (Server) שרת DB-יצירת תוכן ושמירה ב
- (ייצור) PostgreSQL (פיתוח) או SQLite מעל SQLAlchemy ORM: מסד נתונים
 - (server-to-server): נקרא מצד השרת בלבד — OpenAI API: שירות חיצוני
 - Audit Logger: Thread הבקשה לא מעכב את תהליך הבקשה

תיאור הטכנולוגיה הרלוונטית

הסבר	טכנולוגיה	שכבה
שפה מונחית עצמים עם אקוסיסטם עשיר	Python 3.10+	שפת שרת
לבניית יישומי ווב Micro-framework	Flask 3	מסגרת ווב
(SQL) שכבת הפשטה למסד נתונים	Flask-SQLAlchemy	ORM
Postgres; לפיתוח מקומי SQLite לייצור	SQLite / PostgreSQL	מסד נתונים
remember-me ניהול סשן ו	Flask-Login	אימות משתמשים
token CSRF, ולידציה	Flask-WTF, WTForms	טפסים + אבטחה
`generate_password_hash` / `check_password_hash`	Werkzeug Security	הצפנת סיסמאות
Chat Completions API (gpt-4o-mini)	OpenAI Python SDK	AI
XSS עם סינון HTML-המרה ל	markdown + bleach	Markdown
WSGI server (Linux)	Gunicorn	שרת ייצור
PaaS עם HTTPS, Postgres, auto-deploy	Render.com	פריסה
פרוטוקול שרת-לקוח	HTTP/HTTPS	תקשורת

תיאור מודולים בהם נעשה שימוש

מודולים מיובאים

תפקיד	מודול
sessions, ניתוב, בקשות	`flask`
DB לטבלאות Python ממפה מחלקות — ORM	`flask_sqlalchemy`
`@login_required` decorator, ניהול משתמש מחובר	`flask_login`
CSRF + יצירת וולידציית טפסים	`flask_wtf` / `wtforms`
סיסמאות Hash/verify	`werkzeug.security`
OpenAI Chat Completions לקריאת SDK	`openai`
HTML Markdown המרת	`markdown`
XSS מניעת — HTML (whitelist) סינון	`bleach`
Jinja ל-safe-סימון מחרוזת כ	`markupsafe`
env-טעינת משתני סביבה מ	`python-dotenv`
סריאליזציה/דה-סריאליזציה של תשובות שאלון	`json`
rotations כתיבת לוגים לקובץ עם	`logging` + `RotatingFileHandler`
Audit Worker — עבודה אסינכרונית	`threading` (Queue, Thread, Event)
לכל רשומה (UUIDs) יצירת מזהים ייחודיים	`uuid`
UTC חותמות זמן	`datetime`
(Regex) ולידציית חוק סיסמה	`re`
ליצור WSGI שרת	`gunicorn`
Driver ל-PostgreSQL	`psycopg2-binary`
ולידציית כתובות אימייל	`email-validator`

מודלים שלי שבשימוש

`app/models.py` `User` מחלקת

תיאור: מייצגת משתמש רשום במערכת

שדה	סוג	תיאור
id	String(36)	UUID (Primary Key)
email	String(255)	(unique, indexed) כתובת אימייל
password_hash	String(255)	(Hash) סיסמה מוצפנת
first_name	String(50)	שם פרטי (אופציונלי)
last_name	String(50)	שם משפחה (אופציונלי)
created_at	DateTime	(UTC) תאריך יצירה
last_login	DateTime	תאריך התחברות אחרונה

מתודה	טענת כניסה	טענת יציאה
<code>set_password(password: str)</code>	מחרוזת סיסמה גולמית	לערך מוצפן <code>password_hash</code> עדכון
<code>check_password(password: str) → bool</code>	מחרוזת סיסמה לבדיקה	אחרת <code>False</code> , אם תואמת <code>True</code>

`app/models.py` `SkinProfile` מחלקת

תיאור: תשובות שאלון העור + סיכום קצר

שדה	סוג	תיאור
id	String(36)	UUID
user_id	String(36)	FK → users.id (unique)
questionnaire_json	Text	JSON-כל תשובות השאלון כ
summary_label	String(200)	טקסט קצר לתצוגה ("עור יבש — אקנה (י"יובש")
updated_at	DateTime	תאריך עדכון אחרון

מתודה	טענת כניסה	טענת יציאה
<code>get_answers() → dict</code>	—	עם כל תשובות השאלון dict
<code>set_answers(data: dict)</code>	של תשובות dict	questionnaire_json ב JSON שמירת

`app/models.py` `Routine` מחלקת

תיאור: שגרת בוקר/ערב נוכחית של המשתמש

שדה	סוג	תיאור
id	String(36)	UUID
user_id	String(36)	FK → users.id (unique)
morning_text	Text	(Markdown) טקסט שגרת בוקר
evening_text	Text	(Markdown) טקסט שגרת ערב
generated_at	DateTime	מועד יצירה
used_openai	Boolean	Fallback או AI (True) האם נוצרה ע"י (False)

`app/models.py` `RoutineVersion` מחלקת

תיאור: היסטוריית גרסאות — כל שגרה שנוצרה נשמרת

שדה	סוג	תיאור
id	String(36)	UUID

user_id	String(36)	FK → users.id
version_number	Integer	מספר גרסה (3, 2, 1...)
morning_text, evening_text	Text	שגרת בוקר/ערב
generated_at	DateTime	מועד יצירה
used_openai	Boolean	AI או Fallback
answers_json	Text	של התשובות בזמן היצירה JSON עותק

מחלקת `CommunityShare` (app/models.py)

תיאור: פוסט פומבי — עותק של פרופיל ושגרה בזמן השיתוף

שדה	סוג	תיאור
id	String(36)	UUID
user_id	String(36)	FK → users.id
display_name	String(120)	שם תצוגה שנבחר
profile_summary	Text	עותק טקסט של הפרופיל
morning_routine	Text	עותק שגרת בוקר
evening_routine	Text	עותק שגרת ערב
created_at	DateTime	מועד פרסום

מחלקת `CommunityReaction` (app/models.py)

תיאור: תגובת לייק/דיסלייק של משתמש על שיתוף

שדה	סוג	תיאור
id	String(36)	UUID
share_id	String(36)	FK → community_shares.id
user_id	String(36)	FK → users.id
value	Integer	1=like, -1=dislike
created_at, updated_at	DateTime	חתימות זמן

משתמש יכול לתת רק תגובה אחת לכל שיתוף — (share_id, user_id) על UniqueConstraint: **אילוץ**

שירות `routine\_ai.py` (app/services/)

פונקציות ציבוריות:

פונקציה	טענת כניסה	טענת יציאה
generate_routine(answers: dict)	תשובות שאלון dict	tuple: (morning_text, evening_text, used_openai: bool)

פונקציות פנימיות:

פונקציה	תיאור
_build_fallback_routine(answers)	בונה שגרה מובנית לפי כללים
_format_answers_for_prompt(a)	Prompt-לטקסט ל dict ממיר
_split_morning_evening(text)	לשני חלקים (בוקר/ערב) AI מפצל תשובת

שירות `audit\_logger.py` (app/services/)

פונקציות ציבוריות:

פונקציה	תיאור
init_async_audit_logger(app)	נקרא פעם אחת בהפעלת (Worker Thread-מפעיל את ה (האפליקציה

שולח אירוע לתור (לא חוסם)	<code>log_audit_event(event_type, level, **data)</code>
---------------------------	---

(app/) `utils.py` שירות

פונקציה	תיאור
<code>render_markdown_safe(text)</code>	Markup ומחזיר Bleach → HTML → Markdown ממיר בטוח לתצוגה

תיאור סביבת הפיתוח

פירוט	רכיב
Windows 10/11	מערכת הפעלה (פיתוח)
Linux (Render)	מערכת הפעלה (ייצור)
Python 3.10+	שפת תכנות
VS Code / JetBrains	סביבת פיתוח (IDE)
pip + requirements.txt	ניהול חבילות
venv (Python virtual environment)	ניהול סביבות
Git + GitHub	Version Control
Render.com (Blueprint / Web Service)	פריסה
Flask built-in (debug=True)	שרת פיתוח
Gunicorn (WSGI)	שרת ייצור

תיאור האלגוריתמים המרכזיים בפרויקט

(generate_routine) אלגוריתם 1: יצירת שגרה

?לא זמין AI-הדילמה: איך ליצור שגרה חכמה אבל להבטיח שתמיד תהיה תוצאה גם אם ה

— הפתרון: דו-שלבי

- 50. מותאם Prompt עם OpenAI-ניסיון ראשון: קריאה ל.
- 51. אם נכשל — בניית שגרה מבוססת כללים: (Fallback) ניסיון שני.

תיאור:

```
function generate_routine(answers):
    if no API key or OpenAI SDK missing:
        return fallback_routine(answers)

    prompt = build_prompt(answers)
    try:
        response = openai.chat.completions.create(prompt)
        morning, evening = split_response(response)
        if morning and evening are valid:
            return (morning, evening, True)
        else:
            return fallback_routine(answers)
    except any error:
        return fallback_routine(answers)
```

הגישה API-עשירה אך לא רצינו שהמערכת "תיפול" אם יש בעיה ב AI למה בחרנו בגישה זו: רצינו חוויית הדו-שלבית מבטיחה 100% זמינות עבור המשתמש.

split_morning_evening) (לבוקר/ערב AI אלגוריתם 2: פיצול תשובת

הדילמה: מודל השפה מחזיר טקסט אחד ארוך — איך מפצלים אותו נכון

הפתרון:

52. (Markdown כותרות) ## פיצול לפי סימני

53. (לילה " → ערב"/"evening"/"בוקר " ; ערב " → "morning"/"בוקר") לפי מילות מפתח chunk זיהוי כל

54. (פנימי fallback) אם הפיצול נכשל — חצי על חצי

Audit Logger: אלגוריתם 3

הדילמה: כתיבת לוגים לדיסק עלולה להאט את הבקשה. איך לתעד בלי להשפיע על ביצועים

הפתרון: Producer-Consumer pattern:

- Main thread (producer): שם אירוע ב Queue.
- Worker thread (consumer): מוציא מהתור וכותב לקובץ
- Worker thread כ daemon — ייסגר אוטומטית כשהתהליך הראשי נגמר

Toggle: אלגוריתם 4

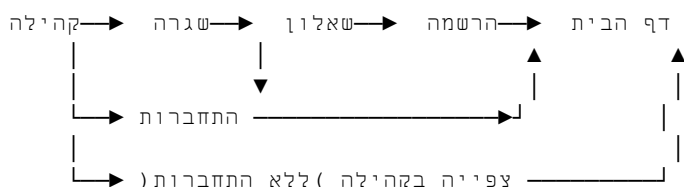
שניתן לחוץ שוב לביטול או להחלפה like/dislike הדילמה: איך לנהל

הפתרון:

```
function react(share_id, user_id, value):
    existing = find_reaction(share_id, user_id)
    if not existing:
        create new reaction(value)
    elif existing.value == value:
        delete existing // toggle off
    else:
        existing.value = value // switch like↔dislike
```

תיאור מסכי הפרויקט

זרימה כללית



מסך	URL נתיב	תיאור
דף הבית	'/'	קישורים להרשמה/התחברות/קהילה . אם מחובר — הפנייה לשאלון
הרשמה	'/register'	טופס: אימייל, שם, סיסמה, אימות
התחברות	'/login'	טופס: אימייל + סיסמה
שכחתי סיסמה	'/forgot-password'	טופס: אימייל + סיסמה חדשה + אימות
שאלון עור	'/questionnaire'	sections שדות מחולקים ל-3 18 (מאפייני עור, אורח חיים, טיפוח נוכחי)
שגרה	'/routine'	Markdown + הצגת שגרת בוקר/ערב ב מספר גרסה

פיד שיתופים + טופס שיתוף + לייק/דיסלייק	/community	קהילה
---	------------	-------

פרוטוקול התקשורת

(ייצור) HTTPS (או HTTP) מעל Request-Response המערכת עובדת במודל

תשובה	גוף הבקשה	URL	Method	פעולה
HTML (או redirect)	—	/	GET	דף הבית
HTML form	—	/register	GET	הצגת טופס הרשמה
redirect → /questionnaire	form-data: email, password, ... + CSRF	/register	POST	שליחת הרשמה
redirect → /questionnaire	form-data: email, password + CSRF	/login	POST	התחברות
redirect → /login	form-data: email, password + CSRF	/forgot-password	POST	איפוס סיסמה
HTML form (pre-filled if exists)	—	/questionnaire	GET	הצגת שאלון
redirect → /routine	form-data: 18 שדות + CSRF	/questionnaire	POST	שליחת שאלון
HTML (Markdown rendered)	—	/routine	GET	צפייה בשגרה
HTML feed	—	/community	GET	פיד קהילה
redirect → /community	form-data: display_name + CSRF	/community/share	POST	שיתוף
redirect → /community	CSRF	/community/react/<id>/<action>	POST	תגובה
redirect → /login	—	/logout	GET	התנתקות

אבטחת פרוטוקול

- hidden field בתוך CSRF token חייבת לכלול POST כל בקשת
- SECRET_KEY) חתום Cookie סשן מנוהל ע"י
- מונע ציתות לתעבורה HTTPS (TLS) — בייצור

מבני נתונים

טבלאות במסד הנתונים

מפתחות זרים	מפתח ראשי	תיאור	טבלה
—	id (UUID)	משתמשים	users
user_id → users	id (UUID)	פרופילי עור	skin_profiles
user_id → users	id (UUID)	שגרה נוכחית	routines
user_id → users	id (UUID)	היסטוריית גרסאות	routine_versions
user_id → users	id (UUID)	שיתופים	community_shares
share_id → community_shares, user_id → users	id (UUID)	לייקים/דיסלייקים	community_reactions

(questionnaire_json) של תשובות שאלון JSON מבנה

```
{
  "skin_type": "oily",
  "age_range": "26_35",
  "concerns": ["acne", "pores"],
  "sensitivities": ["fragrance"],
  "goals": ["acne_control", "hydration"],
  "budget": "medium",
  "time_morning": 10,
  "time_evening": 15,
  "climate": "hot_dry",
  "water_hard": "hard",
  "sun_exposure": "moderate",
  "makeup_frequency": "sometimes",
  "spf_habit": "always",
  "exfoliation": "1_2",
  "retinol": "beginner",
  "pregnancy": "no",
  "uses_actives": true,
  "notes": ""
}
```

Audit Log (JSON per line) מבנה אירוע

```
{
  "timestamp": "2026-05-01T10:30:00+00:00",
  "event_type": "auth.login_success",
  "level": "info",
  "user_id": "abc-123-...",
  "email": "user@example.com"
}
```

פרק ה' – הקוד

(routine_ai.py – generate_routine) קטע 1: יצירת שגרה

```
def generate_routine(answers: dict[str, Any]) -> tuple[str, str, bool]:
    """OpenAI טקסט בוקר, טקסט ערב, האם נוצר עם) מחזיר: """
    api_key = os.environ.get("OPENAI_API_KEY", "").strip()
    if not api_key or OpenAI is None:
        return _build_fallback_routine(answers)

    client = OpenAI(api_key=api_key)
    payload = _format_answers_for_prompt(answers)
    system = (
        "אתה יועץ קוסמטולוגיה להסבר כללי בלבד. אינך מחליף רופא או רוקח. "
        "קצרות, רשימות מחוספרות markdown השב בעברית בלבד. פורמט ברור עם כותרות "
        "ציון סוגי מוצרים) לא מותגים ספציפיים (ושלבים לבוקר ולערב בנפרד"
    )
    user_msg = f"""לפי נתוני המשתמש הבאים, בנה שגרת טיפוח מפורטת:"{payload}"""
```

```

try:
    resp = client.chat.completions.create(
        model=os.environ.get("OPENAI_MODEL", "gpt-4o-mini"),
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": user_msg},
        ],
        temperature=0.5,
    )
    text = (resp.choices[0].message.content or "").strip()
    morning, evening = _split_morning_evening(text)
    if not morning or not evening:
        return _build_fallback_routine(answers)
    return morning, evening, True
except Exception:
    return _build_fallback_routine(answers)

```

זהו הלב של Fallback-ואם נכשלת מכל סיבה — חוזרת ל OpenAI **הסבר**: הפונקציה מנסה ליצור שגרה עם המערכת.

קטע 2: Audit Logger – Worker Thread

```

_queue: Queue[dict[str, Any]] = Queue()

def _worker(app: Flask) -> None:
    logger = _build_logger()
    with app.app_context():
        while not _stop_event.is_set():
            try:
                payload = _queue.get(timeout=0.5)
            except Empty:
                continue
            try:
                logger.info(json.dumps(payload, ensure_ascii=False))
            except Exception:
                app.logger.exception("Failed writing audit event")

def log_audit_event(event_type: str, level: str = "info", **data: Any) -> None:
    payload = {"timestamp": _utc_now(), "event_type": event_type, "level": level,
**data}
    _queue.put(payload)

```

שולחת אירוע לתור ולא מחכה 'log_audit_event' — כל קריאה ל Thread + Queue-**הסבר**: שימוש ב שולף ברציפות מהתור וכותב לקובץ Worker-לכתיבה לדיסק. ה.

קטע 3: ולידציית סיסמה

```

def _validate_password_strength(form, field):
    value = field.data or ""
    if len(value) < 8:
        raise ValidationError("הסיסמה חייבת להכיל לפחות 8 תווים.")
    if not re.search(r"[A-Za-z]", value):
        raise ValidationError("הסיסמה חייבת להכיל לפחות אות אחת באנגלית.")
    if not re.search(r"\d", value):
        raise ValidationError("הסיסמה חייבת להכיל לפחות מספר אחד.")

```

```
if not re.search(r"^[A-Za-z0-9]", value):
    raise ValidationError("הסיסמה חייבת להכיל לפחות תו מיוחד אחד")
```

הסבר: פונקציית ולידציה מותאמת שמאפשרת מדיניות סיסמה חזקה: לפחות 8 תווים, אות באנגלית, מספר, ותו מיוחד.

קטע 4: XSS prevention בטוח Markdown

```
def render_markdown_safe(text: str | None) -> Markup:
    if not text:
        return Markup("")
    raw = markdown.markdown(text, extensions=["nl2br", "sane_lists"],
output_format="html")
    clean = bleach.clean(raw, tags=_allowed_tags(), strip=True)
    return Markup(clean)
```

שלא ברשימה HTML שמסנן כל תגית Bleach) שגרה, פוסטים (עובר דרך Markdown **הסבר:** כל תוכן או המשתמש הכניס תגיות זדוניות AI-גם אם מודל ה XSS המותרת — כך נמנע

קטע 5: Toggle בקהילה

```
@community_bp.route("/community/react/<share_id>/<action>", methods=["POST"])
@login_required
def react_to_share(share_id: str, action: str):
    value = 1 if action == "like" else -1 if action == "dislike" else 0
    reaction = CommunityReaction.query.filter_by(share_id=share_id,
user_id=current_user.id).first()
    if reaction is None:
        reaction = CommunityReaction(share_id=share_id, user_id=current_user.id,
value=value)
        db.session.add(reaction)
    elif reaction.value == value:
        db.session.delete(reaction) # toggle off
    else:
        reaction.value = value # switch
    db.session.commit()
```

חכם — לחיצה ראשונה יוצרת תגובה, לחיצה שנייה על אותו כפתור מבטלת, לחיצה על Toggle **הסבר:** מנגנון הכפתור הנגדי מחליפה.

פרק ו' – בדיקות

עבר/נכשל	תוצאה בפועל	תוצאה מצופה	שם הבדיקה	מספר
		הפנייה לשאלון + DB-משתמש חדש ב	הרשמה תקינה	1
		הודעת שגיאה, לא נוצר משתמש	סיסמה חלשה	2
		הודעת "אימיל או "סיסמה שגויים	התחברות שגויה	3
		הפנייה לשאלון	התחברות נכונה	4
		שגרה נוצרת ומוצגת	מילוי שאלון מלא	5
		הודעת שגיאה, לא	שאלון חלקי	6

		נשמר		
7	AI יצירת שגרה עם	used_openai=True		
8	AI ללא Fallback	used_openai=False, שגרה תקינה		
9	שיתוף בקהילה	פוסט מופיע בפיד		
10	לייק/דיסלייק	ספירה מתעדכנת עובד Toggle		
11	CSRF הגנת	token בקשה ללא נדחית		
12	Audit Log	אירועים נרשמים לקובץ		

פרק ז' – מדריך למשתמש

הדרכה למשתמש הרוצה לקבל שגרת טיפוח

55. <https://personal-skin.onrender.com>: לדוגמה) גישה לאתר — פתח את הדפדפן ועבור לכתובת האתר.
56. הרשמה — לחץ על "הרשמה", "הכנס אימייל, שם) (אופציונלי (וסיסמה חזקה 8+ תווים, אות, מספר, תו "מיוחד. (לחץ "הרשמה
57. מילוי שאלון — ענה על כל השאלות בכנות: סוג עור, גיל, חששות, רגישויות, מטרות, תקציב, זמן, אקלים "ויעוד. לחץ "שליחה ויצירת שגרה מותאמת
58. צפייה בשגרה — תראה שגרת בוקר וערב מותאמת. ניתן לחזור לשאלון ולעדכן בכל עת.
59. "שיתוף בקהילה — אם רוצים — גלול לדף "קהילה, "מלא שם תצוגה ולחץ "פרסמי.

הדרכה למנהל המערכת (מפעיל)

60. התקנה מקומית:

```
cd personal_skin
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
py -3 -m pip install -r requirements.txt
```

61. הגדרת .env: העתק: .env.example ל-.env, הגדר FLASK_SECRET_KEY ו-OPENAI_API_KEY (אופציונלי).
62. הפעלה: `py -3 run.py` — http://127.0.0.1: 5000-השרת עולה ב
63. עצירה: Ctrl+C.
64. הגדר משתני סביבה, Render (Blueprint)-חבר ל, GitHub-פריסה לייצור: דחוף ל.

פרק ח' – מבט אישי

(לסיכום אישי — מומלץ לכתוב על: מה למדת? אילו קשיים עלו? מה היית עושה אחרת? תודות)

פרק ט' – ביבליוגרפיה

65. Flask Documentation — <https://flask.palletsprojects.com/>
66. Flask-SQLAlchemy — <https://flask-sqlalchemy.palletsprojects.com/>
67. Flask-Login — <https://flask-login.readthedocs.io/>
68. Flask-WTF — <https://flask-wtf.readthedocs.io/>
69. WTForms — <https://wtforms.readthedocs.io/>
70. OpenAI API Reference — <https://platform.openai.com/docs/api-reference>
71. Werkzeug Security — <https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security>
72. Python markdown — <https://python-markdown.github.io/>
73. Bleach (HTML sanitizer) — <https://bleach.readthedocs.io/>
74. Render.com Docs — <https://render.com/docs>
75. Gunicorn — <https://gunicorn.org/>
76. Python threading — <https://docs.python.org/3/library/threading.html>

נספח – הקוד

GitHub-הקוד המלא של הפרויקט זמין ב

«*GitHub Repository* קישור»: *הכנס קישור ל*

קבצים עיקריים:

תיאור	קובץ
נקודת כניסה	<code>run.py</code>
(Gunicorn) נקודת כניסה לייצור	<code>wsgi.py</code>
Blueprints יצירת אפליקציה, רישום	<code>app/__init__.py</code>
הגדרות לפי סביבה	<code>app/config.py</code>
SQLAlchemy + LoginManager	<code>app/extensions.py</code>
(User, SkinProfile, Routine, ...) כל המודלים	<code>app/models.py</code>
הרשמה, התחברות, איפוס, התנתקות	<code>app/auth/routes.py</code>
שאלון + יצירת שגרה	<code>app/questionnaire/routes.py</code>
פיד, שיתוף, תגובות	<code>app/community/routes.py</code>
(AI + Fallback) יצירת שגרה	<code>app/services/routine_ai.py</code>
Audit Log	<code>app/services/audit_logger.py</code>
Markdown safe filter	<code>app/utils.py</code>
(קבצים 7) HTML תבניות	<code>templates/</code>
CSS עיצוב	<code>static/css/style.css</code>
Python חבילות	<code>requirements.txt</code>
Render-הגדרות פריסה ל	<code>render.yaml</code>